

HW1-9.2

邓若辰 PB25111617

1.7

1. 通过 `scanf` 和 `printf` 语句可以显式地表示程序的输入和输出。对于简单程序，其可读性较强；但对于复杂程序，代码不易修改，并且某个模块产生的错误可能传播到其他模块。
 2. 通过函数及其参数传递，可以实现部分代码的复用。
 3. 通过函数调用和参数传递，各模块能够按照统一的接口相互调用，从而降低模块之间的耦合，并将错误尽可能限制在模块内部。
-

1.12

已知：

$$f(n) = 21n^4 + n^2 + 1000$$

$$g(n) = 15n^4 + 500n^3$$

$$h(n) = 5000n^{3.5} + n \log n$$

(1) $f(n) = O(g(n))$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{21n^4 + n^2 + 1000}{15n^4 + 500n^3} = \frac{21}{15} = \frac{7}{5}$$

正确。

(2) $f(n) = O(h(n))$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{h(n)} = \lim_{n \rightarrow \infty} \frac{21n^4 + n^2 + 1000}{5000n^{3.5} + n \log n} = \infty$$

错误。

(3) $g(n) = O(h(n))$

$$\lim_{n \rightarrow \infty} \frac{g(n)}{h(n)} = \lim_{n \rightarrow \infty} \frac{15n^4 + 500n^3}{5000n^{3.5} + n \log n} = \infty$$

错误。

(4) $h(n) = O(n^{3.5})$

$$\lim_{n \rightarrow \infty} \frac{h(n)}{n^{3.5}} = \lim_{n \rightarrow \infty} \left(5000 + \frac{n \log n}{n^{3.5}} \right) = 5000$$

正确。

(5) $h(n) = O(n \log n)$

$$\lim_{n \rightarrow \infty} \frac{h(n)}{n \log n} = \lim_{n \rightarrow \infty} \left(\frac{1000n^{3.5}}{n \log n} + 1 \right) = \infty$$

错误。

1.17

k 阶 Fibonacci 数列求第 m 项

```
long long fib(int k, int m)
{
    if (m < 0)
    {
        return 0;
    }

    if (m < k - 1)
    {
        return 0;
    }

    if (m == k - 1)
    {
        return 1;
    }

    long long sum = 0;

    for (int i = 1; i <= k; i++)
    {
        sum += fib(k, m - i);
    }

    return sum;
}
```

该递归算法的时间复杂度为指数级：

$$O(k^m)$$

递归调用的空间复杂度为：

$$O(m)$$

2.12

比较两个顺序表 La 和 Lb 。去除最大公共前缀之后：

- 若均不为空，则首个元素较大的顺序表较大；
- 若其中一个不为空，长度较大的顺序表较大；
- 若均为空，则两个顺序表相等。

据题意，不妨把问题化简为：

两个顺序表 La 和 Lb 。从第一个元素开始依次比较：

- 如果对应位置的元素不相等，则元素较大的顺序表较大；
- 如果公共部分的元素全部相等，则长度较大的顺序表较大；
- 如果元素和长度均相同，则两个顺序表相等。

返回值：

- -1 : $La < Lb$;
- 0 : $La = Lb$;
- 1 : $La > Lb$ 。

```
int compare(Sqlist La, Sqlist Lb)
{
    int i = 0;

    while (i < La.length && i < Lb.length)
    {
        if (La.elem[i] < Lb.elem[i])
        {
            return -1;
        }

        if (La.elem[i] > Lb.elem[i])
        {
            return 1;
        }

        i++;
    }

    if (La.length < Lb.length)
    {
        return -1;
    }

    if (La.length > Lb.length)
    {
        return 1;
    }
}
```

```
    return 0;  
}
```

即利用同一个下标同时控制两个线性表，且舍去了复杂的最大公共前缀求解环节

设两个顺序表的长度分别为 m 和 n ，则时间复杂度为：

$$O(\min(m, n))$$

空间复杂度为：

$$O(1)$$